INTEL IMAGE CLASSIFICATION

Using Convolutional Neural Networks (CNN)
with MobileNetV2 Transfer Learning

PROJECT DOCUMENTATION

Project	Intel Image Classification using CNN
Dataset	Intel Image Classification — Kaggle
Framework	TensorFlow / Keras
Platform	Google Colab
Submitted By	Shafaat Ullah
Department	Artificial Intelligence
Institute	Freelancing Hub Gilgit
Submission	October 2026

A practical deep-learning project for classifying natural and urban scene images into six categories.

Contents

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Project Objectives
- 5. Dataset Description
- 6. Tools and Technologies
- 7. Proposed Methodology
- 8. CNN and Transfer Learning Architecture
- 9. Training Configuration
- 10. Evaluation Metrics
- 11. Results and Output Files
- 12. Limitations
- 13. Future Improvements
- 14. Conclusion
- 15. References

Note: The final measured test accuracy and loss are intentionally not pre-filled. Insert the values printed by Cell 12 after training; reporting unmeasured results would be inaccurate.

1. Abstract

This project implements an image-classification system using a convolutional neural network (CNN) with MobileNetV2 transfer learning. The system categorizes scene photographs into six classes: buildings, forest, glacier, mountain, sea, and street. The workflow is designed for Google Colab and uses TensorFlow/Keras to load images, preprocess data, train a classification head, fine-tune selected pretrained layers, and evaluate predictions on a held-out test set. Evaluation outputs include test accuracy and loss, a confusion matrix, a per-class classification report, sample predictions, and a CSV file containing predicted labels and confidence scores.

2. Introduction

Image classification is a computer-vision task in which a model assigns an image to one of a set of predefined categories. CNNs are widely used because convolutional filters learn spatial patterns such as edges, textures, shapes, and higher-level visual features. Training a deep model from scratch can require substantial data and computing resources. Transfer learning addresses this challenge by reusing visual features learned by a model pretrained on a large image dataset.

3. Problem Statement

The goal is to build an automated system that recognizes the scene category of an input image. Manual classification can be slow and inconsistent at scale. The project explores whether a CNN with pretrained visual features can classify common natural and urban scenes and provide measurable, reproducible evaluation outputs.

4. Project Objectives

- Download the dataset programmatically from Kaggle using KaggleHub.
- Load and resize images into a consistent 160 × 160 pixel format.
- Create training and validation subsets while retaining a separate test set.
- Train a MobileNetV2-based classifier using TensorFlow/Keras.
- Fine-tune selected pretrained layers with a low learning rate.
- Evaluate performance using accuracy, loss, precision, recall, F1-score, and a confusion matrix.
- Export the trained model, visualizations, and per-image predictions.

5. Dataset Description

The project uses the Intel Image Classification dataset hosted on Kaggle by the dataset publisher Puneet6060. The commonly used archive provides training and testing directories, along with a prediction-image directory. The six scene labels are listed below. The notebook discovers the training and testing folders by checking for the expected class directories rather than relying on a fixed absolute path.

Class label	Example scene type
buildings	Buildings and built-up areas
forest	Forested landscapes
glacier	Glacier and ice-covered landscapes
mountain	Mountainous landscapes
sea	Sea and coastal water scenes
street	Roads and urban street scenes

Data split: 20% of the provided training directory is used for validation; the provided test directory is reserved for final evaluation. Exact sample counts depend on the downloaded dataset version.

6. Tools and Technologies

Technology	Purpose
Python	Programming language and notebook workflow
Google Colab	Cloud-based execution environment
KaggleHub	Programmatic dataset download
TensorFlow / Keras	Dataset loading, model construction, training, and evaluation
MobileNetV2	Pretrained convolutional feature extractor
NumPy and Pandas	Numerical processing and prediction export
Matplotlib	Training graphs and prediction visualizations
scikit-learn	Confusion matrix and classification metrics

7. Proposed Methodology

Step 1 — Dataset acquisition

KaggleHub downloads the dataset directly into the Colab environment. The notebook locates the training and testing folders and verifies that the expected six class directories are present.

Step 2 — Image loading and preparation

Images are resized to 160 × 160 pixels and grouped into batches of 32. The training directory is split into 80% training and 20% validation subsets. The test directory is loaded separately and with shuffling disabled so that labels remain aligned with prediction rows.

Step 3 — Data augmentation

Random horizontal flips, small rotations, and zoom transformations are applied during training to expose the model to modest variations. Augmentation is not used to alter the held-out test images.

Step 4 — Transfer learning

MobileNetV2 is initialized with ImageNet pretrained weights and its original classification head is removed. A global average pooling layer, dropout layer, and six-unit softmax output layer are added for the scene classes.

Step 5 — Initial training and fine-tuning

First, the pretrained feature extractor is frozen and the new classification head is trained for eight epochs. Next, the last 30 base-model layers are made trainable, with batch-normalization layers kept frozen, and the model is fine-tuned for up to seven additional epochs at a lower learning rate. A checkpoint callback saves the model with the best validation accuracy, and a learning-rate scheduler reduces the learning rate when validation loss plateaus.

Step 6 — Evaluation and export

The best checkpoint is loaded and evaluated on the test set. Predicted class indices are compared with ground-truth labels, then the model, graphs, confusion matrix, and prediction CSV are saved to the Colab /content directory.

8. CNN and Transfer Learning Architecture

Layer / component	Role
Input: 160 × 160 × 3	Receives RGB scene images
Data augmentation	Random flip, rotation, and zoom during training
MobileNetV2 preprocessing	Scales input pixels to the format expected by pretrained MobileNetV2
MobileNetV2 base	Extracts hierarchical visual features learned from ImageNet
Global average pooling	Converts spatial feature maps into a compact feature vector
Dropout (0.30)	Regularizes the classification head
Dense softmax (6 units)	Returns a probability distribution over six classes

Output interpretation: the softmax value with the highest probability is selected as the predicted class. The confidence percentage is the selected softmax probability multiplied by 100; it is not a guarantee that the prediction is correct.

9. Training Configuration

Parameter	Configuration
Image size	160 × 160 pixels, RGB
Batch size	32
Optimizer	Adam
Initial learning rate	0.001
Fine-tuning learning rate	0.00001
Loss function	Sparse categorical cross-entropy
Primary metric	Accuracy
Epochs	8 initial + up to 7 fine-tuning epochs
Checkpoint selection	Highest validation accuracy

10. Evaluation Metrics

Accuracy: The fraction of test images whose predicted class matches the true class.

Precision: For a class, the fraction of images predicted as that class that are truly members of it.

Recall: For a class, the fraction of all true examples of that class that the model correctly identifies.

F1-score: The harmonic mean of precision and recall, balancing both measures.

Confusion matrix: A table comparing true labels with predicted labels to show which categories are correctly classified and which are confused.

Loss: The categorical cross-entropy objective used to measure how well predicted class probabilities match the true labels.

11. Results and Output Files

The notebook prints the measured test accuracy and test loss after evaluation. Since the model has not been run within this documentation-generation step, no numerical performance claim is fabricated. Fill the result summary below using the values from Cell 12 after a successful run.

Result	Measured value
Test accuracy	Enter value from Cell 12: _____
Test loss	Enter value from Cell 12: _____
Per-class precision / recall / F1	Use classification report from Cell 13

Files produced by the notebook

- **intel_image_classification_cnn.keras** — saved trained model.
- **intel_cnn_accuracy_loss.png** — training and validation accuracy/loss curves.
- **intel_cnn_confusion_matrix.png** — test-set confusion matrix.
- **intel_cnn_predictions.csv** — actual class, predicted class, confidence percentage, and correctness for each test image.
- **intel_cnn_sample_predictions.png** — sample image predictions with confidence scores.

12. Limitations

- Performance depends on the downloaded dataset version, hardware, training duration, and runtime state.
- A high softmax confidence does not guarantee a correct or calibrated prediction.
- The six labels describe scene categories; the model is not designed for object detection or pixel-level segmentation.
- The checkpoint is selected using validation accuracy, so final test performance may differ.
- The saved model includes a preprocessing Lambda layer; when loading it, the notebook supplies the corresponding preprocess_input function through custom_objects.

13. Future Improvements

- Compare MobileNetV2 against a custom CNN and other pretrained architectures.
- Use class-wise error analysis to identify visually similar categories.

- Tune image size, batch size, dropout, augmentation strength, and learning rate.
- Add a small inference interface that accepts an uploaded image and displays its predicted class.
- Track experiments and preserve dataset/version details for reproducibility.

14. Conclusion

This project provides an end-to-end image-classification workflow using TensorFlow/Keras and MobileNetV2 transfer learning. It covers dataset acquisition, preprocessing, augmentation, model training, fine-tuning, evaluation, visualization, and export. The resulting model can predict one of six scene categories for a new image. Final conclusions about performance should be based on the actual test metrics produced by the completed notebook.

15. References

- Kaggle dataset: Intel Image Classification — Puneet6060.
- TensorFlow/Keras documentation: https://www.tensorflow.org/.
- MobileNetV2 API: TensorFlow MobileNetV2 documentation.
- KaggleHub: KaggleHub GitHub repository.

End of Project Documentation